Evaluation plan

Project: Auditable AI Agents
Status: plan. Section 7 says what is already measured; everything else is still to do.

---

1. Research questions (locked in `docs/proposal/decisions.md`)

1. Does the verifier detect every edit, delete, insert, reorder and truncation attack on an anchored log, and can it locate the first altered entry?
2. What is the cost (gas) versus delay trade-off as the anchoring batch size changes?
3. How much latency do logging and anchoring add to each agent step?
4. How do verification time and storage scale from 100 to 100,000 entries?

---

2. Threat model

Actor	Can	Cannot
Honest agent operator	Run the agent, store logs	n/a
Dishonest operator or attacker with file access	Edit, delete, insert, reorder or truncate the stored log; recompute any hash they like	Change a root already written to the blockchain
Blockchain	Keep roots unchanged and timestamped	Know whether a log entry was true
Out of scope	Compromised logger recording false entries before anchoring; compromised agent key; denial of service; a 51 percent attack on the chain; production smart contract audit	

Key consequence: the system gives **tamper evidence after anchoring**, not tamper prevention and not truthfulness.

---

3. Experiment 1: detection (RQ1)

Setup. Generate logs of 2 to 40 entries; anchor them with a random batch size; apply one attack; run the verifier.

Attacks. Edit a payload; edit a timestamp; edit a stored hash; delete an entry; insert a duplicate; swap two entries; truncate; and (separately) the consistent rewrite, where every later hash is recomputed.

Metrics. Detection rate (target 100 percent); false-positive rate on untouched logs (target 0 percent); localisation: the entry index reported equals the edited index for in-place edits, and the batch range for consistent rewrites.

Status. Done in `tests/test_fuzz_tamper.py` (300 trials per attack, fixed seeds).

---

4. Experiment 2: gas versus batch size (RQ2)

Setup. Anchor a log of N = 1,000 entries with batch sizes B in {1, 2, 5, 10, 25, 50, 100, 500, 1000} on the local Hardhat chain.

Measure. Gas per `anchor()` call; total gas for the log; gas per entry; **delay**: the number of entries (and seconds at the agent's step rate) that sit unprotected before the next anchor, which is at most B - 1 entries.

Output. A table and a plot of gas per entry versus B, with the delay on a second axis.

Expectation to test, not assume. Total gas should fall roughly as 1/B because each call has a fixed cost; per-batch gas should be nearly constant in B because only one root is stored.

Money. Convert gas to a currency figure with a stated gas price and ether price, labelled as an illustration, since the local chain has no real cost.

---

5. Experiment 3: overhead per agent step (RQ3)

Setup. Run the same task 30 times in each condition: (a) agent without logging; (b) with hash-chain logging only; (c) with logging and anchoring every B = 5 entries. Use the scripted model (to isolate logging cost) and qwen3:8b (to show cost relative to real model latency).

Measure. Wall-clock time per step and per task; report median and interquartile range.

Expected finding to check. Logging cost is negligible next to model latency; anchoring cost is dominated by chain round trips.

---

6. Experiment 4: verification scaling (RQ4)

Setup. Synthetic logs of N in {100, 1,000, 10,000, 100,000} entries with a 5-entry payload; B = 100.

Measure. Time to verify the hash chain; time to recompute all batch roots; total verification time; memory; size of the log file; bytes stored on-chain (should be constant per batch).

Also. Time to verify one entry by inclusion proof versus whole-log verification, and the proof size.

---

7. What is already measured

Item	Result so far	Where
Detection of 7 attack types	300 of 300 trials per type detected	tests/test_fuzz_tamper.py
False alarms on untouched logs	0 of 300	same
Python and Solidity hash agreement	All proofs for trees of 1, 2, 3, 5, 8, 13 and 100 entries verify on-chain	chain-tests/AuditAnchor.test.js

Item	Result so far	Where
Gas per anchor call (single demo run)	About 79,500 to 93,100	scripts/run_demo.py output; the single-run figures are not yet a study
Real local model run	qwen3:8b completed a 12-entry run; it anchored and verified	tests/test_ollama_live.py, demo

---

8. Reporting rules

- Report medians with ranges, not single runs; fix seeds and record versions (Python, Node, Hardhat, solc, Ollama model digest).
- Keep raw results in `research/experiments/` as JSON and generate every figure from them with a script.
- Say what failed or surprised. A measured cost that is worse than expected is still a result.